Master of Business Analytics

Prescriptive Analytics & Optimization

MBAN 5540

Week 10: Dynamic Programming

Instructor: Majid Taghavi

Sobey School of Business
Saint Mary's University

Overview

- Dynamic Programming (DP) is a sequential procedure for optimizing an objective function

Objective	
Maximization	Minimization
Reward Function	Cost Function

- Main features:
  - The underlying problem can be broken down into stages (such as time periods).
  - The cost function is additive over time.

Background

- We start by reviewing some background concepts. This includes:
  - Recursion
  - Proof by induction
  - Memoization

Recursion

- When a function calls itself until a base case is reached.

- Example: Factorial
  - Base case: $0! = 1$

$$n! = n \times (n-1) \times (n-2) \times (n-3) \times 3 \times 2 \times 1$$

$$n! = n \times (n-1)!$$

Proof by induction

- Proof by induction:
  1. Prove the base case, i.e., $n = 0$ or 1
  2. Prove that if the statement holds for k, it will hold for $k + 1$

- Example: Sum of n Numbers

$$1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}$$

$$1^3 + 2^3 + 3^3 + \cdots + n^3 = \left[\frac{n(n+1)}{2}\right]^2$$

Fibonacci Numbers

- In Fibonacci sequence, each number is the sum of the two preceding ones.

- The base case:

$$F(0) = 0$$
$$F(1) = 1$$

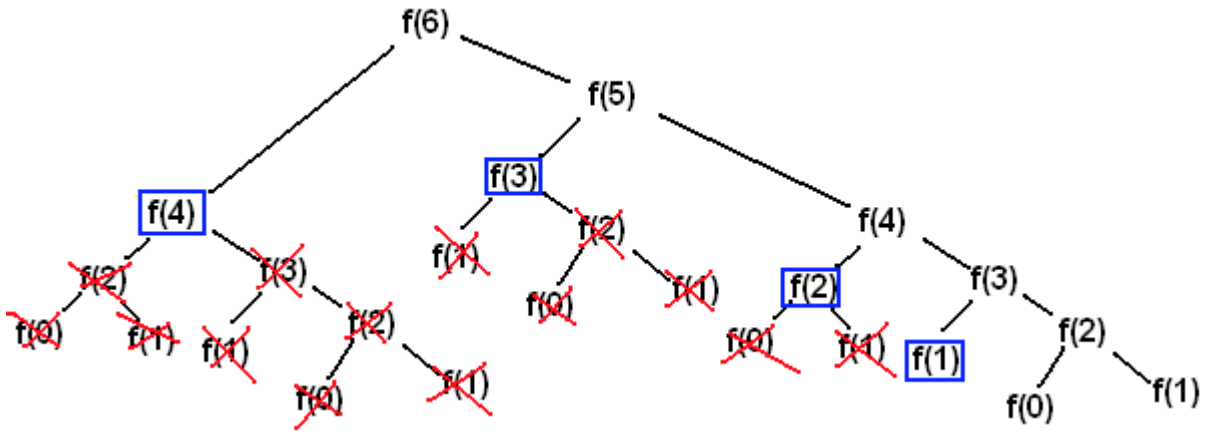
- Recursive Relationship:

$$F(n) = F(n-1) + F(n-2)$$

Memoization

- Memoization: is a technique used to speed up running time of codes by storing the results of function calls that might occur again.

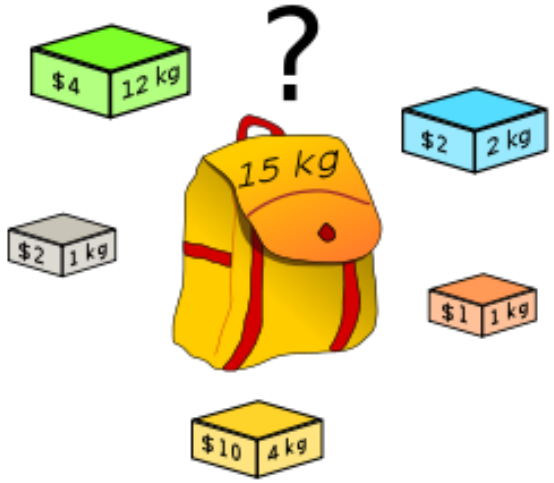
- To this end, usually a data structure is needed to store the results



DP is in fact using recursion with Memoization!

The Knapsack Problems (revisit)

- Suppose we have a set of items:
  - Each item has a value and a weight



- The goal is to maximize the total value that we put in our knapsack

- The knapsack has a given capacity.

- We solved knapsack models using integer programming before. Can we use DP to solve them?

The Knapsack Problems

- Does knapsack problem have the necessary characteristics to be solved using dynamic programming?
  - Stages
  - Recursive relationship

- Base case:
  - No capacity available ($c = 0$)
  - No items available ($n = 0$)

- For capacity c:

$$F_i(c) = \max_{x_i}[v_i x_i + F_{i-1}(c - w_i x_i)\ ]$$

Bellman's Principle of Optimality

- An optimal policy has the property that whatever the initial state and initial decision are, the remaining decisions must constitute an optimal policy with regard to the state resulting from the first decision

- From any point on an optimal trajectory, the remaining trajectory is optimal for the corresponding problem over the remaining number of stages or time interval initiated at that point.

- A problem is said to satisfy the Principle of Optimality if the sub-solutions of an optimal solution of the problem are themselves optimal solutions for their subproblems.

Steps to Construct a DP Model

- State the objective function
- Define the base case
- Develop the recursive relationship that identifies the optimal policy
- Find where the final answer will be.

Characteristics of DP Problems

- The problem can be divided into **stages**, with a **policy decision** required at each stage.
- Each stage has several **states** associated with the beginning of this stage.
- The effect of the policy decision at each state is to transform the current state to a state associated with the beginning of the next stage.
- The solution procedure is designed to find an **optimal policy** for the overall problem, i.e., a prescription of the optimal policy decision at each stage for each of the possible states.

Characteristics of DP Problems

- Given the current state, an optimal policy for the remaining stages is independent of the policy decisions adopted in the previous stages. Therefore, the optimal immediate decision depends on only the current state and not on how you got there. This is the principle of optimality for dynamic programming.

- A recursive relationship that identifies the optimal policy for stage n given the optimal policy for stage n + 1 is available.

- The solution procedure begins by finding the optimal policy for the last stage and moving backwards stage by stage, each time finding the optimal policy for that stage until it finds the optimal policy starting at the initial stage.

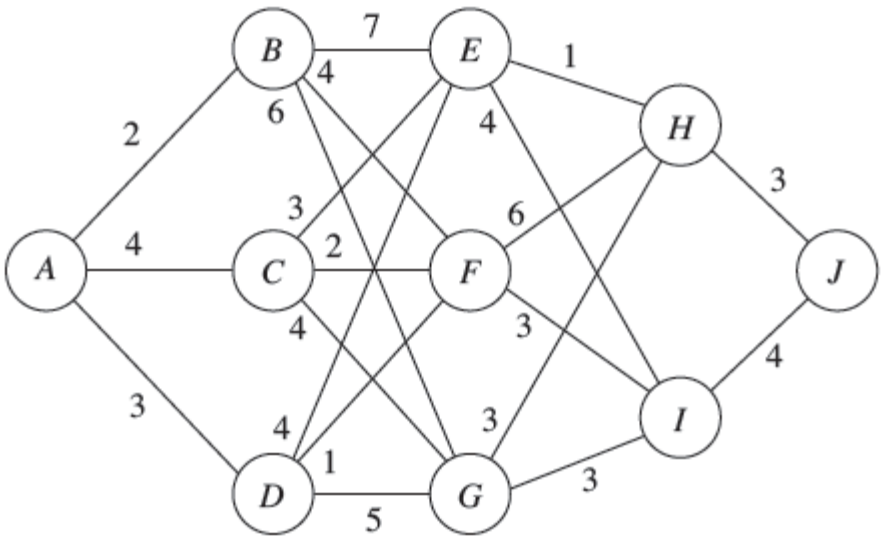
DP Notation

- N: number of stages
- n: current stage ($n = 1, 2, \cdots, N$)
- s_n: current state for stage n
- x_n: decision variable for stage n
- x_n^*: optimal value of x_n given s_n
- $f_n(s_n, x_n)$: contribution of stages $n, n+1, \cdots, N$ to objective function if system starts in state s_n at stage n.

$$f_n^*(s_n) = \underset{x_n}{Min}\{f_n(s_n, x_n)\} \text{ or } \underset{x_n}{Max}\{f_n(s_n, x_n)\}$$

Shortest Path Problem

- What are the stages?



- What are the states in each stage?

- How to define decision variables?
- x_n $(n = 1, 2, 3, 4)$ is the immediate decision on stage n

Shortest Path Problem

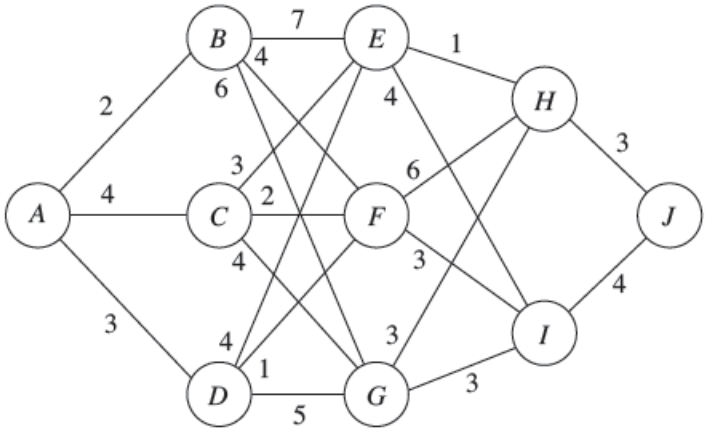
- What is the recursive relationship?

$f_n(s, x_n)$ = cost of stage n (immediate) + minimum future costs for stages $n+1$ onward

$$f_n(s, x_n) = c_{sx_n} + f^*_{n+1}(x_n)$$

Stage: $n = 4$		
s	$\boldsymbol{f^*_4(s)}$	$\boldsymbol{x^*_4}$
H	3	J
I	4	J

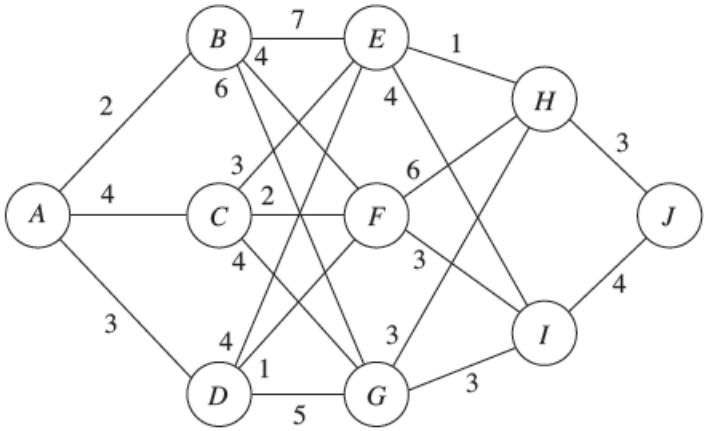
Stage: n =3				
s	$f_3(s, x_3) = c_{3x_3} + f^*_4(x_3)$		$\boldsymbol{f^*_3(s)}$	$\boldsymbol{x^*_3}$
	H	I		
E	1+3	4+4	4	H
F	6+3	3+4	7	I
G	3+3	3+4	6	H



Shortest Path Problem

Stage: n =2					
s	$f_2(s, x_2) = c_{2x_2} + f_3^*(x_2)$			$\boldsymbol{f_2^*(s)}$	$\boldsymbol{x_2^*}$
	E	F	G		
B	7+4	4+7	6+6	11	E or F
C	3+4	2+7	4+6	7	E
D	4+4	1+7	5+6	8	E or F

Stage: n =1					
s	$f_1(s, x_1) = c_{1x_1} + f_2^*(x_1)$			$\boldsymbol{f_1^*(s)}$	$\boldsymbol{x_1^*}$
	B	C	D		
A	2+11	4+7	3+8	11	C or D



$A \to C \to E \to H \to J$ or $A \to D \to E \to H \to J$ or $A \to D \to F \to I \to J$

Shortest Path Algorithms

- Dijkstra's Algorithm (greedy)
- Bellman-Ford (DP)
- Floyd Warshall (DP)